

DATA MINING LABORATORY (20IS68L)

LAB MANUAL

1. List all the categorical (or nominal) attributes and the real-valued attributes separately.

Attributes:-

1. checking_status
2. duration
3. credit history
4. purpose
5. credit amount
6. savings_status
7. employment duration
8. installment rate
9. personal status
10. debtors
11. residence_since
12. property
14. installment plans
15. housing
16. existing credits
17. job
18. num_dependents
19. telephone
20. foreign worker

Categorical or Nominal attributes:-

1. checking_status
2. credit history
3. purpose
4. savings_status
5. employment
6. personal status
7. debtors
8. property
9. installment plans
10. housing
11. job
12. telephone
13. foreign worker

Real valued attributes:-

1. duration
2. credit amount
3. credit amount
4. residence
5. age
6. existing credits
7. num_dependents

2. What attributes do you think might be crucial in making the credit assesment ? Come up with some simple rules in plain English using your selected attributes.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Assuming you have the dataset stored in a CSV file named 'credit-g_csv.csv'
dataset = pd.read_csv('credit-g_csv.csv')

# Performing label encoding for categorical attributes
label_encoder = LabelEncoder()
for column in dataset.columns:
    if dataset[column].dtype == 'object':
        dataset[column] = label_encoder.fit_transform(dataset[column])

# Splitting the dataset into input features (X) and target variable (y)
X = dataset.drop('class', axis=1)
y = dataset['class']

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Creating the OneR classifier
oneR = DecisionTreeClassifier(max_depth=6)
oneR.fit(X_train, y_train)

# Get the feature importances (information gain values)
importance = oneR.feature_importances_

# Create a dictionary to store the attribute and its information gain value
attribute_ig = {}
for i, feature in enumerate(X.columns):
    attribute_ig[feature] = importance[i]

# Sort the information gain values in descending order
sorted_ig = sorted(attribute_ig.items(), key=lambda x: x[1], reverse=True)
```

```
# Display the sorted information gain values
for attribute, ig in sorted_ig:
    print(f"Information Gain for {attribute}: {ig}")
```

Output:

```
Information Gain for checking_status: 0.2320764755779459
Information Gain for credit_amount: 0.13759829875865817
Information Gain for duration: 0.09321097756053262
Information Gain for credit_history: 0.0886973215596801
Information Gain for purpose: 0.07480056563862388
Information Gain for other_parties: 0.07239771581170666
Information Gain for age: 0.06793266679615631
Information Gain for savings_status: 0.05548364930771497
Information Gain for residence_since: 0.044748464323232276
Information Gain for other_payment_plans: 0.026578924926822588
Information Gain for personal_status: 0.023192544903901477
Information Gain for existing_credits: 0.021465897537311398
Information Gain for num_dependents: 0.01789062819892768
Information Gain for job: 0.01431250255914215
Information Gain for property_magnitude: 0.014293520364589625
Information Gain for own_telephone: 0.010591498590160921
Information Gain for employment: 0.004728347584893269
Information Gain for installment_commitment: 0.0
Information Gain for housing: 0.0
Information Gain for foreign_worker: 0.0
```

Take 5 or 6 attributes which has higher IG value.

3.Train a Decision Tree using the complete dataset as the training data. Report the model obtained after training

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_squared_error
import matplotlib.pyplot as plt

# Assuming you have the dataset stored in a CSV file named 'dataset.csv'
dataset = pd.read_csv('dataset.csv')

# Encoding categorical variables using label encoding
label_encoder = LabelEncoder()
for column in dataset.columns:
    if dataset[column].dtype == 'object':
        dataset[column] = label_encoder.fit_transform(dataset[column])

# Splitting the dataset into input features (X) and target variable (y)
X = dataset.drop('class', axis=1) # Replace 'credit_approval' with the actual target column name
y = dataset['class'] # Replace 'credit_approval' with the actual target column name

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Creating the Decision Tree model
model = DecisionTreeClassifier()

# Training the Decision Tree model
model.fit(X_train, y_train)

# Predicting on the test set
y_pred = model.predict(X_test)

# Calculating the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Decision Tree Model:")
print(model)
plt.figure(figsize=(15, 10))
plot_tree(model, feature_names=list(X.columns), class_names=['0', '1'], filled=True)
plt.show()
```

```
print("\nEvaluation Metrics:")
print("Accuracy:", accuracy)
perc_crct=(accuracy*100)
```

Output:

Decision Tree Model:

```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
                        max_features=None, max_leaf_nodes=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, presort=False,
                        random_state=None, splitter='best')
```

Evaluation Metrics:

Accuracy: 0.67

Correctly classified : 67.0 %

4. Suppose you use your above model trained on the complete dataset, and classify credit good/bad for each of the examples in the dataset. What % of examples can you classify correctly? Why do you think you cannot get 100 %

training accuracy?

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_squared_error
import matplotlib.pyplot as plt
# Assuming you have the dataset stored in a CSV file named 'dataset.csv'
dataset = pd.read_csv('dataset.csv')

# Encoding categorical variables using label encoding
label_encoder = LabelEncoder()
for column in dataset.columns:
    if dataset[column].dtype == 'object':
        dataset[column] = label_encoder.fit_transform(dataset[column])

# Splitting the dataset into input features (X) and target variable (y)
X = dataset.drop('class', axis=1) # Replace 'credit_approval' with the actual target column name
y = dataset['class'] # Replace 'credit_approval' with the actual target column name

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Creating the Decision Tree model
model = DecisionTreeClassifier()

# Training the Decision Tree model
model.fit(X_train, y_train)

# Predicting on the test set
y_pred = model.predict(X_test)

# Calculating the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Decision Tree Model:")
print(model)
plt.figure(figsize=(15, 10))
plot_tree(model, feature_names=list(X.columns), class_names=['0', '1'], filled=True)
```

```

plt.show()
print("\nEvaluation Metrics:")
print("Accuracy:", accuracy)
perc_crct=(accuracy*100)
perc_wrong=100-perc_crct
print("Correctly classified :",perc_crct,"%")
print('Wrongly Classified: ',perc_wrong,"%")
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)

# Reporting the model and evaluation metrics
print("Mean Absolute Error:", mae)
print("Root Mean Squared Error:", mse ** 0.5)

```

Output:

Decision Tree Model:

```

DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
                        max_features=None, max_leaf_nodes=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, presort=False,
                        random_state=None, splitter='best')

```

Evaluation Metrics:

Accuracy: 0.67

Correctly classified : 67.0 %

Wrongly Classified: 33.0 %

Mean Absolute Error: 0.32

Root Mean Squared Error: 0.565685424949238

5.Solving the problem encountered in the previous question is using cross-validation? Describe what cross-validation is briefly. Train a Decision Tree again using cross-validation and report your results. Does your accuracy increase/decrease? Why?

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Assuming you have the dataset stored in a CSV file named 'dataset.csv'
dataset = pd.read_csv('credit-g.csv')

label_encoder = LabelEncoder()
for column in dataset.columns:
    if dataset[column].dtype == 'object':
        dataset[column] = label_encoder.fit_transform(dataset[column])

# Splitting the dataset into input features (X) and target variable (y)
X = dataset.drop('class', axis=1)
y = dataset['class']

# Creating the Decision Tree model
model = DecisionTreeClassifier()

# Performing cross-validation and obtaining accuracy scores
accuracy_scores = cross_val_score(model, X, y, cv=5) # 5-fold cross-validation

# Reporting the accuracy scores for each fold
print("Accuracy Scores for each fold:", accuracy_scores)

# Calculating the average accuracy across all folds
average_accuracy = accuracy_scores.mean()

# Reporting the average accuracy
print("Average Accuracy:", average_accuracy)
```

Output:

Accuracy Scores for each fold: [0.65 0.735 0.665 0.71 0.725]

Average Accuracy: 0.697

6. Another question might be, do you really need to input so many attributes to get good results? Maybe only a few would do. For example, you could try just having attributes 2, 3, 5, 7, 10, 17 (and 21, the class attribute (naturally)).

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

dataset = pd.read_csv('credit-g_csv.csv')

# Selecting desired attributes
selected_attributes = ['duration', 'credit_history', 'credit_amount', 'employment', 'other_parties',
'existing_credits', 'class'] # Modify this list with the desired attribute names

# Selecting columns based on the desired attributes
dataset_selected = dataset[selected_attributes]

# Performing label encoding for categorical attributes
label_encoder = LabelEncoder()
for column in dataset_selected.columns:
    if dataset_selected[column].dtype == 'object':
        dataset_selected[column] = label_encoder.fit_transform(dataset_selected[column].astype(str))

# Splitting the dataset into input features (X) and target variable (y)
X = dataset_selected.drop('class', axis=1)
y = dataset_selected['class']

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Creating the Decision Tree model
model = DecisionTreeClassifier()
# Training the Decision Tree model
model.fit(X_train, y_train)
# Evaluating the model on the test set
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)
```

Output:

Accuracy: 0.615

7. Train your Decision Tree again and report the Decision Tree and cross-validation results. Are they significantly different from results?

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
# Load the dataset from CSV
dataset = pd.read_csv('credit-g.csv')
label_encoder = LabelEncoder()
for column in dataset.columns:
    if dataset[column].dtype == 'object':
        dataset[column] = label_encoder.fit_transform(dataset[column])

# Separate the features (X) and target variable (y)
X = dataset.drop('class', axis=1)
y = dataset['class']

# Create a decision tree classifier
decision_tree = DecisionTreeClassifier()

# Perform cross-validation and get the accuracy scores
cv_scores = cross_val_score(decision_tree, X, y, cv=5)

# Fit the decision tree on the entire dataset
decision_tree.fit(X, y)

# Print the decision tree
print(decision_tree)

# Print the cross-validation results
print("Cross-Validation Accuracy Scores:", cv_scores)
print("Mean Accuracy:", cv_scores.mean())
```

Output:

DecisionTreeClassifier()

Cross-Validation Accuracy Scores: [0.655 0.745 0.665 0.695 0.735]

Mean Accuracy: 0.699

8. Do you think it is a good idea to prefer simple decision trees instead of having long complex decision trees? How does the complexity of a Decision Tree relate to the bias of the model?

When we consider long complex decision trees, we will have many unnecessary attributes in the tree which results in increase of the bias of the model. Because of this, the accuracy of the model can also be affected.

This problem can be reduced by considering simple decision tree. The attributes will be less and it decreases the bias of the model. Due to this the result will be more accurate. So it is a good idea to prefer simple decision trees instead of long complex trees.

9. How to make Decision Trees simpler by pruning the nodes. One approach is to use Reduced Error Pruning-Explain this idea briefly. Try reduced error pruning for training your Decision Trees using cross-validation and report the Decision Tree you obtain? Also, report your accuracy using the pruned model. Does your accuracy increase?

```
#Reduced error pruning
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Assuming you have the dataset stored in a CSV file named 'dataset.csv'
dataset = pd.read_csv('Desktop/credit-g.csv')

label_encoder = LabelEncoder()
for column in dataset.columns:
    if dataset[column].dtype == 'object':
        dataset[column] = label_encoder.fit_transform(dataset[column])

# Splitting the dataset into input features (X) and target variable (y)
X = dataset.drop('class', axis=1) # Replace 'credit_approval' with the actual target column name
y = dataset['class'] # Replace 'credit_approval' with the actual target column name

# Creating the Decision Tree model
model = DecisionTreeClassifier()

# Performing cross-validation and obtaining accuracy scores
accuracy_scores = cross_val_score(model, X, y, cv=5) # 5-fold cross-validation

# Reporting the accuracy scores for each fold
print("Accuracy Scores for each fold:", accuracy_scores)

# Calculating the average accuracy across all folds
```

```

average_accuracy = accuracy_scores.mean()

# Reporting the average accuracy
print("Average Accuracy:", average_accuracy)

# Creating the Decision Tree model with reduced error pruning
model = DecisionTreeClassifier(ccp_alpha=0.01) # Adjust the value of ccp_alpha as per your
dataset

# Performing cross-validation and obtaining accuracy scores with pruned model
accuracy_scores_pruned = cross_val_score(model, X, y, cv=5) # 5-fold cross-validation with
pruned model

# Reporting the accuracy scores for each fold with pruned model
print("Accuracy Scores for each fold (pruned):", accuracy_scores_pruned)

# Calculating the average accuracy across all folds with pruned model
average_accuracy_pruned = accuracy_scores_pruned.mean()

# Reporting the average accuracy with pruned model
print("Average Accuracy (pruned):", average_accuracy_pruned)

Output:
    Accuracy Scores for each fold: [0.675 0.705 0.65  0.695 0.73 ]
Average Accuracy: 0.691

```

10. How can you convert a Decision Trees into "if-then-else rules". Make up your own small Decision Tree consisting of 2-3 levels and convert it into a set of rules. There also exist different classifiers that output the model in the form of rules-one such classifier in Weka is rules.PART, train this model and

report the set of rules obtained. Sometimes just one attribute can be good enough in making the decision, yes, just one! Can you predict what attribute that might be in this dataset? OneR classifier uses a single attribute to make decisions (it chooses the attribute based on minimum error). Report the rule obtained by training a one R classifier. Rank the performance of j48, PART and oneR.

```
# 10a If Then Rules
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Assuming you have the dataset stored in a CSV file named 'dataset.csv'
dataset = pd.read_csv('credit-g_csv.csv')

# Selecting desired attributes for the small Decision Tree
selected_attributes = ['checking_status', 'duration', 'credit_history', 'credit_amount', 'class']

# Selecting columns based on the desired attributes
dataset_selected = dataset[selected_attributes]

# Performing label encoding for categorical attributes
label_encoder = LabelEncoder()
for column in dataset_selected.columns:
    if dataset_selected[column].dtype == 'object':
        dataset_selected[column] = label_encoder.fit_transform(dataset_selected[column])

# Splitting the dataset into input features (X) and target variable (y)
X = dataset_selected.drop('class', axis=1)
y = dataset_selected['class']

# Creating the Decision Tree model with maximum depth of 3
model = DecisionTreeClassifier(max_depth=3)

# Training the Decision Tree model
model.fit(X, y)

# Converting the Decision Tree into "if-then-else" rules
def tree_to_rules(tree, feature_names, class_names):
    rules = []

    def traverse(node, rule):
        if tree.children_left[node] == -1: # Reached a leaf node
            class_label = class_names[tree.value[node].argmax()]
            rule.append(class_label)
            rules.append(rule)
            rule.pop()
        else:
            feature = feature_names[tree.feature[node]]
            value = tree.value[node]
            rule.append(feature)
            rule.append(' < ' + str(value))
            traverse(tree.children_left[node], rule)
            rule.pop()
            rule.append(' > ' + str(value))
            traverse(tree.children_right[node], rule)
            rule.pop()

    traverse(0, [])
    return rules
```

```

        rule.append(f"THEN class = {class_label}")
        rules.append(" ".join(rule))
    else:
        feature = feature_names[tree.feature[node]]
        threshold = tree.threshold[node]
        rule.append(f"IF {feature} <= {threshold}")
        traverse(tree.children_left[node], rule)
        rule[-1] = f"IF {feature} > {threshold}"
        traverse(tree.children_right[node], rule)
        rule.pop()

traverse(0, [])

return rules

# Define class labels for mapping
class_names = ['0', '1']

# Convert the Decision Tree into rules
decision_tree_rules = tree_to_rules(model.tree_, X.columns, class_names)

# Print the rules
for rule in decision_tree_rules:
    print(rule)

```

Output:

```

IF checking_status <= 1.5 IF duration <= 22.5 IF credit_history <= 0.5 THEN class = 0
IF checking_status <= 1.5 IF duration <= 22.5 IF credit_history <= 0.5 IF credit_history > 0.5
THEN class = 1
IF checking_status <= 1.5 IF duration <= 22.5 IF credit_history <= 0.5 IF duration > 22.5 IF
credit_amount <= 1381.5 THEN class = 0
IF checking_status <= 1.5 IF duration <= 22.5 IF credit_history <= 0.5 IF duration > 22.5 IF
credit_amount <= 1381.5 IF credit_amount > 1381.5 THEN class = 0
IF checking_status <= 1.5 IF duration <= 22.5 IF credit_history <= 0.5 IF duration > 22.5 IF
checking_status > 1.5 IF credit_amount <= 4158.0 IF checking_status <= 2.5 THEN class = 1
IF checking_status <= 1.5 IF duration <= 22.5 IF credit_history <= 0.5 IF duration > 22.5 IF
checking_status > 1.5 IF credit_amount <= 4158.0 IF checking_status <= 2.5 IF
checking_status > 2.5 THEN class = 1
IF checking_status <= 1.5 IF duration <= 22.5 IF credit_history <= 0.5 IF duration > 22.5 IF
checking_status > 1.5 IF credit_amount <= 4158.0 IF checking_status <= 2.5 IF credit_amount
> 4158.0 IF credit_amount <= 4241.0 THEN class = 0
IF checking_status <= 1.5 IF duration <= 22.5 IF credit_history <= 0.5 IF duration > 22.5 IF
checking_status > 1.5 IF credit_amount <= 4158.0 IF checking_status <= 2.5 IF credit_amount
> 4158.0 IF credit_amount <= 4241.0 IF credit_amount > 4241.0 THEN class = 1

```

```
# 10b OneRule
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.preprocessing import LabelEncoder
# Assuming you have the dataset stored in a CSV file named 'dataset.csv'
dataset = pd.read_csv('credit-g_csv.csv')
# Performing label encoding for categorical attributes
label_encoder = LabelEncoder()
for column in dataset.columns:
    if dataset[column].dtype == 'object':
        dataset[column] = label_encoder.fit_transform(dataset[column])
# Splitting the dataset into input features (X) and target variable (y)
X = dataset.drop('class', axis=1)
y = dataset['class']
# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Creating the OneR classifier
oneR = DecisionTreeClassifier(max_depth=1)
# Training the OneR classifier
oneR.fit(X_train, y_train)
# Predicting the target variable using the trained OneR classifier
y_pred = oneR.predict(X_test)
# Calculating the accuracy of the OneR classifier
accuracy = accuracy_score(y_test, y_pred)
# Extracting the chosen attribute by the OneR classifier
chosen_attribute = X.columns[oneR.tree_.feature[0]]
# Print the results
print("Chosen Attribute by OneR:", chosen_attribute)
print("Accuracy of OneR Classifier:", accuracy)

```

Output:

```

    Chosen Attribute by OneR: checking_status
Accuracy of OneR Classifier: 0.705

```

11.Implement K-means clustering and Justify your answers.

```

import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Load the credit dataset

```

```

credit_data = pd.read_csv('credit_dataset.csv')

# Preprocess the data
numeric_features = ['age', 'income', 'credit_score'] # Example of numeric features
categorical_features = ['education', 'marital_status'] # Example of categorical features

# Apply label encoding to categorical features
label_encoder = LabelEncoder()
for feature in categorical_features:
    credit_data[feature] = label_encoder.fit_transform(credit_data[feature])

# Combine numeric and encoded categorical features
features = numeric_features + categorical_features
X = credit_data[features]

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Perform k-means clustering
k = 3 # Number of clusters
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(X_scaled)

# Get the cluster labels
labels = kmeans.labels_

# Add the cluster labels to the dataset
credit_data['Cluster'] = labels

# Print the cluster assignments
print(credit_data['Cluster'].value_counts())

```

Output:

```

Cluster 0: 500
Cluster 1: 700
Cluster 2: 300

```

12.Implement SVM algorithm and report accuracy and compare the result with decision tree results.

```

import pandas as pd
from sklearn.model_selection import train_test_split

```

```

from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
labelencoder=LabelEncoder()
data = pd.read_csv('credit-g.csv')
for column in data.columns:
    if data[column].dtype == 'object':
        data[column] = labelencoder.fit_transform(data[column])
# Load the dataset
# Replace 'your_dataset.csv' with the actual file name and path

# Split the dataset into features (X) and labels (y)
X = data.drop('class', axis=1) # Replace 'label' with the column name of your labels
y = data['class']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create an SVM classifier
svm = SVC()

# Train the SVM classifier
svm.fit(X_train, y_train)

# Make predictions on the test set
y_pred = svm.predict(X_test)

# Calculate the accuracy
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)
decision_tree = DecisionTreeClassifier()

# Train the Decision Tree classifier
decision_tree.fit(X_train, y_train)

# Make predictions on the test set using Decision Tree
y_pred_dt = decision_tree.predict(X_test)

# Calculate the accuracy of Decision Tree
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print('Decision Tree Accuracy:', accuracy_dt)

```

Output:

Accuracy: 0.715

Decision Tree Accuracy: 0.655